

**UNIVERSITY OF DHAKA**

**RedGrid:  
Dependency-Constrained UCB  
Exploration for Autonomous  
Penetration Testing**

Submitted by

Class Roll: 50028

Registration No: H-55

Class Roll: 50040

Registration No: H-49

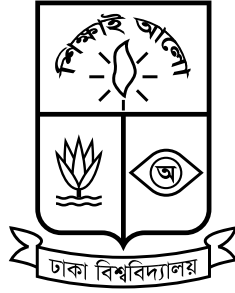
Submitted to the

Department of Computer Science and Engineering

University of Dhaka, Bangladesh

in partial fulfillment of the requirements for the degree of  
Professional Masters in Information and Cyber Security (PMICS).

# Declaration Form



University of Dhaka

We hereby declare that the work in this report titled “**RedGrid: Dependency-Constrained UCB Exploration for Autonomous Penetration Testing**” has been carried out by us under the supervision of Dr. Md. Abdur Razzaque. We further declare that where information has been derived from the literature, we have adequately cited and referenced the original sources. No part of this report was previously presented for another degree or diploma at this or any other institution.

Name of the Candidate

Nishan Paul

Name of the Candidate

Md Rakibur Rahman

Dr. Md. Abdur Razzaque

Professor and Chairman

Dept. of

Computer Science and Engg.

University of Dhaka

*“To conquer fear, you must become fear.”*

Ra’s al Ghul, Batman Begins

# UNIVERSITY OF DHAKA

## *Abstract*

Autonomous penetration testing agents powered by large language models have shown rapid progress in recent years, yet published evaluations consistently point to two recurring weaknesses: agents explore candidate vulnerabilities too narrowly, and they struggle to reason about the prerequisite relationships between different stages of an attack. These weaknesses limit how far current systems can be trusted to operate against realistic, multi-step web, API, and network targets without close human supervision.

This report documents the first stage of an investigation into whether explicitly modeling the dependency relationships between attack steps — rather than treating candidate vulnerabilities as an unordered list — can improve how autonomous agents explore and prioritize their actions during a penetration test. As an initial step, we reviewed a body of recent literature on LLM-based offensive security agents, covering single-agent systems, multi-agent orchestration frameworks, planning-augmented approaches, and the benchmarks used to evaluate them. Two recurrent failure modes — insufficient exploration breadth and weak prerequisite reasoning — were identified across independently developed systems. These observations motivate the research direction described in this report, which we are calling RedGrid.

At this stage, our work is preliminary: a structured review of the current state of autonomous penetration testing research and an early architectural direction motivated by the gaps observed in the literature. The remaining work over the coming months will focus on refining this direction, implementing and evaluating it against established benchmarks, and determining whether dependency-aware exploration meaningfully improves autonomous exploitation performance.

# *Acknowledgements*

We are sincerely grateful to our supervisor, Dr. Md. Abdur Razzaque, Professor and Chairman, Department of Computer Science and Engineering, University of Dhaka, for agreeing to guide this work and for the time he has already invested in it at such an early stage. His feedback pushed us to move beyond simply summarizing existing autonomous penetration testing systems and to instead ask what specific gap in that body of work our own research could realistically address. That question shaped the direction this thesis has taken so far, and we expect it to keep shaping the work as it progresses over the coming months.

We would also like to acknowledge the wider research community whose published work – spanning benchmark design, agent architectures, and empirical evaluations in offensive security – gave us the foundation on which this thesis is being built. Any progress we make over the next six months will rest heavily on the groundwork laid by that literature.

# Contents

|                                                            |             |
|------------------------------------------------------------|-------------|
| <b>Declaration Form</b>                                    | <b>i</b>    |
| <b>Abstract</b>                                            | <b>iii</b>  |
| <b>Acknowledgements</b>                                    | <b>iv</b>   |
| <b>List of Figures</b>                                     | <b>vii</b>  |
| <b>List of Tables</b>                                      | <b>viii</b> |
| <b>1 Introduction</b>                                      | <b>1</b>    |
| 1.1 Background and Motivation . . . . .                    | 1           |
| 1.1.1 Cybersecurity and Vulnerability Assessment . . . . . | 1           |
| 1.1.2 Why Conventional VAPT Has Limits . . . . .           | 2           |
| 1.1.3 Large Language Models and AI Agents . . . . .        | 2           |
| 1.1.4 The Emergence of Autonomous VAPT Research . . . . .  | 3           |
| 1.1.5 Motivation for This Thesis . . . . .                 | 4           |
| 1.2 Problem Statement . . . . .                            | 4           |
| 1.3 Research Objectives . . . . .                          | 6           |
| 1.4 Scope of the Research . . . . .                        | 7           |
| 1.5 Research Direction . . . . .                           | 9           |
| 1.6 Anticipated Challenges . . . . .                       | 10          |
| 1.7 Organisation of This Report . . . . .                  | 11          |
| <b>2 Literature Review and Related Work</b>                | <b>12</b>   |
| 2.1 Domain Overview . . . . .                              | 12          |
| 2.2 Review of Existing Work . . . . .                      | 13          |
| 2.2.1 Early Single-Agent Approaches . . . . .              | 13          |
| 2.2.2 Multi-Agent and Team-Based Orchestration . . . . .   | 15          |
| 2.2.3 Planning-Augmented Approaches . . . . .              | 16          |
| 2.2.4 Non-Web Attack Surfaces and Benchmarks . . . . .     | 16          |
| 2.3 Comparative Analysis . . . . .                         | 17          |
| 2.4 Research Gap . . . . .                                 | 19          |
| 2.5 Summary . . . . .                                      | 20          |
| <b>3 Proposed Methodologies</b>                            | <b>22</b>   |

|          |                                                                                                              |           |
|----------|--------------------------------------------------------------------------------------------------------------|-----------|
| 3.1      | System Overview . . . . .                                                                                    | 22        |
| 3.2      | Requirement Analysis . . . . .                                                                               | 22        |
| 3.3      | Proposed Architecture . . . . .                                                                              | 22        |
| 3.4      | System Workflow . . . . .                                                                                    | 22        |
| 3.5      | Tools and Technologies . . . . .                                                                             | 22        |
| <b>4</b> | <b>Execution Plan</b>                                                                                        | <b>23</b> |
| 4.1      | Work Breakdown Structure . . . . .                                                                           | 23        |
| 4.2      | Project Timeline . . . . .                                                                                   | 23        |
| 4.3      | Milestones . . . . .                                                                                         | 23        |
| 4.4      | Resource Requirements . . . . .                                                                              | 23        |
| 4.5      | Risk Assessment . . . . .                                                                                    | 23        |
| <b>5</b> | <b>Experimental Results</b>                                                                                  | <b>24</b> |
| 5.1      | Evaluation Plan . . . . .                                                                                    | 24        |
| <b>6</b> | <b>Conclusions</b>                                                                                           | <b>25</b> |
| 6.1      | Summary . . . . .                                                                                            | 25        |
| 6.2      | Expected Deliverables . . . . .                                                                              | 25        |
| 6.3      | Future Works . . . . .                                                                                       | 25        |
|          | <b>Bibliography</b>                                                                                          | <b>26</b> |
|          | <b>Appendix A: RedGrid Reference Material</b>                                                                | <b>28</b> |
| .1       | Full VDG Node Schema . . . . .                                                                               | 28        |
| .2       | Ordinal Evidence Score ( $E_{\text{ord}}$ ) Reference . . . . .                                              | 30        |
| .3       | Benchmark Suite Summary . . . . .                                                                            | 30        |
| .4       | UCB Hyperparameter Search Grid . . . . .                                                                     | 31        |
| .5       | Ablation Condition A1 — Precise Pseudocode . . . . .                                                         | 32        |
|          | Condition (b) — UCB with dependency-constrained<br>eligible set (pre-filter): . . . . .                      | 32        |
|          | Condition (c) — Stacked: UCB over all nodes, post-<br>filter by dependency (the key discriminant): . . . . . | 32        |
| .6       | Prior Work Gap Summary . . . . .                                                                             | 33        |

# List of Figures

|     |                                                                                                                                                        |   |
|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------|---|
| 1.1 | Exploration-failure rate as the dominant CVE-Bench failure mode, by agent and mode (data from CVE-Bench’s Table 5 failure-mode breakdown [1]). . . . . | 5 |
|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------|---|



# List of Tables

|     |                                                                                                                                                                                      |    |
|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1.1 | PentestEval ground-truth-injection ablation: marginal gain from replacing each stage’s output with expert ground truth, applied cumulatively on top of the SMP baseline [2]. . . . . | 6  |
| 1.2 | Attack surfaces currently considered in scope, and their candidate governing benchmarks (preliminary). . . . .                                                                       | 8  |
| 2.1 | The eleven papers selected for review, with their main focus and relevance to this research. . . . .                                                                                 | 14 |
| 2.2 | Comparison of systems and benchmarks reviewed, across four recurring architectural dimensions. . . . .                                                                               | 18 |
| 2.3 | Observed split in the reviewed literature between exploration-oriented and dependency-reasoning-oriented systems (preliminary reading). .                                            | 19 |
| 1   | Complete VDG Node Schema . . . . .                                                                                                                                                   | 28 |
| 2   | Ordinal Evidence Score ( $E_{\text{ord}}$ ) Rubric . . . . .                                                                                                                         | 30 |
| 3   | Full REDGRID Benchmark Suite . . . . .                                                                                                                                               | 31 |
| 4   | UCB Hyperparameter Search Grid . . . . .                                                                                                                                             | 32 |
| 5   | Prior Work Gaps and REDGRID Fixes . . . . .                                                                                                                                          | 33 |

# List of Algorithms

| <b>Acronym</b> | <b>What (it) Stands For</b>                                                      |
|----------------|----------------------------------------------------------------------------------|
| <b>ADM</b>     | <b>A</b> ttack <b>D</b> ecision- <b>M</b> aking                                  |
| <b>AI</b>      | <b>A</b> rtificial <b>I</b> ntelligence                                          |
| <b>API</b>     | <b>A</b> pplication <b>P</b> rogramming <b>I</b> nterface                        |
| <b>CVE</b>     | <b>C</b> ommon <b>V</b> ulnerabilities and <b>E</b> xposures                     |
| <b>CVSS</b>    | <b>C</b> ommon <b>V</b> ulnerability <b>S</b> coring <b>S</b> ystem              |
| <b>CTF</b>     | <b>C</b> apture <b>T</b> he <b>F</b> lag                                         |
| <b>DAG</b>     | <b>D</b> irected <b>A</b> cylic <b>G</b> raph                                    |
| <b>EL</b>      | <b>E</b> nvironmental <b>L</b> ayer                                              |
| <b>GPT</b>     | <b>G</b> enerative <b>P</b> re-trained <b>T</b> ransformer                       |
| <b>LLM</b>     | <b>L</b> arge <b>L</b> anguage <b>M</b> odel                                     |
| <b>PDDL</b>    | <b>P</b> lanning <b>D</b> omain <b>D</b> efinition <b>L</b> anguage              |
| <b>RCE</b>     | <b>R</b> emote <b>C</b> ode <b>E</b> xecution                                    |
| <b>ReAct</b>   | <b>R</b> eason + <b>A</b> ct (agent framework)                                   |
| <b>REST</b>    | <b>R</b> epresentational <b>S</b> tate <b>T</b> ransfer                          |
| <b>SQL</b>     | <b>S</b> tructured <b>Q</b> uery <b>L</b> anguage                                |
| <b>SQLi</b>    | <b>SQL</b> Injection                                                             |
| <b>SSRF</b>    | <b>S</b> erver-Side <b>R</b> equest <b>F</b> orgery                              |
| <b>SSTI</b>    | <b>S</b> erver-Side <b>T</b> emplate <b>I</b> njection                           |
| <b>UCB</b>     | <b>U</b> pper <b>C</b> onfidence <b>B</b> ound                                   |
| <b>VAPT</b>    | <b>V</b> ulnerability <b>A</b> ssessment and <b>P</b> enetration <b>T</b> esting |
| <b>VDG</b>     | <b>V</b> ulnerability <b>D</b> ependency <b>G</b> raph                           |
| <b>WAF</b>     | <b>W</b> eb <b>A</b> pplication <b>F</b> irewall                                 |
| <b>XSS</b>     | <b>C</b> ross-Site <b>S</b> cripting                                             |

| Constant Name  | Symbol | = | Constant Value (with units)                          |
|----------------|--------|---|------------------------------------------------------|
| Speed of Light | $c$    | = | $2.997\,924\,58 \times 10^8 \text{ ms}^{-1}$ (exact) |

---

| symbol   | name              | unit                  |
|----------|-------------------|-----------------------|
| $a$      | distance          | m                     |
| $P$      | power             | W (Js <sup>-1</sup> ) |
| $\omega$ | angular frequency | rads <sup>-1</sup>    |

*For/Dedicated to/To my...*

# Chapter 1

## Introduction

### 1.1 Background and Motivation

#### 1.1.1 Cybersecurity and Vulnerability Assessment

Modern organisations depend on networked software systems for nearly every operational function, and the security of those systems has become a central concern. A security vulnerability is a flaw in software, configuration, or design that an attacker can exploit to gain unauthorised access, execute arbitrary code, disrupt service, or exfiltrate data. Vulnerabilities are regularly discovered in web applications, APIs, network services, and operating systems, and the window between discovery and the deployment of a fix can leave a system exposed for days or weeks.

**Vulnerability Assessment and Penetration Testing (VAPT)** is the practice of proactively identifying and demonstrating these weaknesses before an adversary does. A penetration test is an authorised, systematic attempt to discover and exploit weaknesses in a target system, typically following a structured workflow: initial reconnaissance to understand the target’s attack surface, identification of candidate weaknesses, exploitation of confirmed vulnerabilities, and reporting of findings with remediation guidance. VAPT is widely recognised as an essential security practice,

and demand for it has grown substantially as organisations increase their exposure through cloud services, APIs, and web applications.

### 1.1.2 Why Conventional VAPT Has Limits

Despite its importance, conventional penetration testing faces several practical constraints. Skilled human testers are scarce and expensive. A thorough engagement against a moderately complex target can take days of expert effort. Scheduled, periodic testing leaves gaps between assessments during which new vulnerabilities may be introduced. Automated scanning tools — such as network port scanners and web application vulnerability scanners — can partially fill this gap, but they generally operate on fixed rule sets and signature databases, and they lack the contextual reasoning needed to chain together multi-step exploits. They are effective for known, templatable vulnerabilities, but they cannot adapt to novel application logic, reason about prerequisite relationships between attack steps, or operate across the breadth of a modern attack surface without human guidance.

These limitations have motivated a long-standing research interest in automating parts of the VAPT process.

### 1.1.3 Large Language Models and AI Agents

Large Language Models (LLMs) are deep learning models trained on large corpora of text that have demonstrated a remarkable ability to follow instructions, reason through multi-step problems, write and execute code, and interact with external tools. Models such as GPT-4 have achieved strong performance across a wide range of tasks that previously required domain expertise [3].

An **LLM agent** extends a base language model with the ability to take actions in an environment: calling tools (such as a web browser, a terminal, or an API), observing the results, and iterating toward a goal over multiple steps. The simplest agents follow a *Reason + Act* (ReAct) loop — reason about the current state,



decide on an action, observe the outcome, and repeat. More sophisticated designs introduce hierarchical planning, role-specialised sub-agents, persistent memory, and structured validation.

The emergence of capable LLM agents has opened a genuinely new question for cybersecurity: *can an LLM agent carry out penetration testing tasks autonomously, without step-by-step human direction?*

#### 1.1.4 The Emergence of Autonomous VAPT Research

Over the past two years, a growing body of research has answered parts of this question affirmatively. Early work by Fang et al. [4] showed that a single GPT-4 agent with browser access could autonomously exploit simplified web vulnerabilities. A subsequent study by the same group demonstrated that GPT-4, when given a CVE description, could exploit 87% of a set of 15 real-world one-day vulnerabilities — while every other tested model and open-source scanner achieved 0% [5]. Importantly, without the CVE description, GPT-4’s success rate collapsed to 7%, revealing a sharp distinction between the ability to exploit a known vulnerability and the ability to discover one independently.

These results prompted a rapid expansion of the research area. Zhu et al. [6] demonstrated that a hierarchical team of planning and task-specific agents could autonomously exploit real zero-day vulnerabilities — vulnerabilities for which no description is provided. Deng et al. [7] identified a recurring failure mode in single-agent systems: context loss over long sessions, which causes agents to lose track of what they have already tried and repeat unproductive actions. Subsequent work proposed multi-agent architectures, memory-augmented systems, and planning-augmented approaches to address these and related limitations.

A consistent finding across these papers is that **architecture, not model scale, appears to be the dominant variable in autonomous exploitation performance** — a well-structured pipeline running a comparatively inexpensive model

tends to outperform an unstructured, loosely-scaffolded agent loop running a frontier model. This finding, observed independently across six systems in our review, shaped the direction of our own work.

### 1.1.5 Motivation for This Thesis

Our starting point for this work was to understand this body of research well enough to determine whether it had already addressed the core challenges of autonomous VAPT, and if not, where exactly the remaining gaps lie. Over the past several weeks, we assembled and studied a focused set of papers spanning single-agent exploitation studies, hierarchical multi-agent frameworks, planning-augmented approaches, and the principal benchmark suites used to evaluate autonomous VAPT agents (surveyed in full in Chapter 2).

Two recurring failure modes surfaced across independently developed systems, and they are the gaps this thesis has chosen to investigate first. We want to be upfront that at this stage this is a direction we are pursuing and beginning to develop — not a result we have already obtained.

## 1.2 Problem Statement

The literature reviewed during the preparation of this thesis points to two recurring failure modes that appear across independently developed autonomous VAPT systems. Quantitative evidence for both is provided by dedicated benchmark studies that allow the failures to be measured, not merely described. The two modes are not unrelated: they can be understood as two sides of the same underlying problem, which is described after each failure mode.

**Failure Mode 1 – Insufficient exploration.** On CVE-Bench [1], a benchmark of 40 critical (CVSS  $\geq 9.0$ ) real-world web-application CVEs, the strongest reported

agent exploits only 13% of one-day and 10% of zero-day vulnerabilities. CVE-Bench’s own failure-mode breakdown identifies *insufficient exploration* as the dominant cause of failure across every agent and evaluation setting, with exploration-failure rates ranging from 37.5% to 80.0% (T-Agent: 80.0% zero-day / 55.0% one-day; AutoGPT: 72.5% / 45.0%; Cy-Agent: 67.5% / 37.5%), as illustrated in Figure 1.1. Crucially, this does not appear to be a reasoning-quality problem so much as a breadth-of-search problem: agents commit early to a narrow set of candidate attack paths and rarely return to explore the rest of the surface.

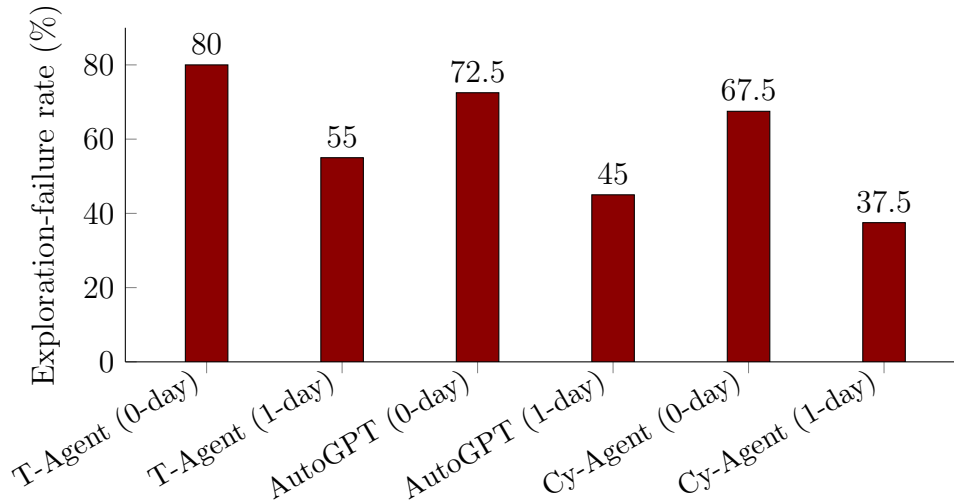


FIGURE 1.1: Exploration-failure rate as the dominant CVE-Bench failure mode, by agent and mode (data from CVE-Bench’s Table 5 failure-mode breakdown [1]).

**Failure Mode 2 – The dependency-reasoning gap.** PentestEval [2] decomposes the pentesting pipeline into six stages – Information Collection (IC), Weakness Gathering (WG), Weakness Filtering (WF), Attack Decision-Making (ADM), Exploit Generation (EG), and Exploit Revision (ER) – and finds the opposite bottleneck at the *planning* level: ADM, which requires reasoning about prerequisite dependencies between candidate weaknesses, is the single lowest-scoring stage (Spearman correlation 0.25 against expert judgement). PentestEval’s ground-truth-injection ablation quantifies roughly how much room for improvement exists at each stage, summarized in Table 1.1.

ADM delivers the largest single-stage marginal increment (+0.14) of the three tested – measured, notably, on top of an already ground-truthed WG+WF pipeline, not

TABLE 1.1: PentestEval ground-truth-injection ablation: marginal gain from replacing each stage’s output with expert ground truth, applied cumulatively on top of the SMP baseline [2].

| Configuration                     | End-to-end success | Marginal gain |
|-----------------------------------|--------------------|---------------|
| SMP (baseline)                    | 0.31               | –             |
| + GT Weakness Gathering (WG)      | 0.50               | +0.19         |
| + GT Weakness Filtering (WF)      | 0.53               | +0.03         |
| + GT Attack Decision-Making (ADM) | 0.67               | <b>+0.14</b>  |

in isolation. Any dependency structure an agent grows on its own, without ground truth, is necessarily a weaker approximation than this, so a realistic ceiling for any such approach would have to sit below 0.67 rather than above it.

**The gap we have not found closed in the reviewed literature.** Systems that target exploration breadth (HPTSA [6], VulnBot [8], and similar team-dispatch architectures) tend to dispatch flat, undifferentiated task lists with no explicit prerequisite or dependency model. Systems that target dependency reasoning (PentestEval’s SMP baseline, CHECKMATE-style classical-planning approaches [9]) are evaluated on curated scenarios with pre-enumerated weakness sets and do not appear to scale to open-ended, wide-surface exploration. Within the papers we reviewed, we did not find a system that combines exploration over a dependency-aware structure, prerequisite relationships grown dynamically from live discovery during a mission, and evaluation across more than one independently-benchmarked attack-surface family. Investigating whether closing this compound gap is possible, and whether doing so is actually worthwhile, is the problem this thesis sets out to explore.

## 1.3 Research Objectives

Guided by the two gaps identified in Section 1.2, we have set out the following working objectives for this thesis. These reflect the research questions we are currently investigating, not a checklist of work already completed. The exact scope

and design of each objective is expected to be refined as implementation and early experiments provide feedback that the literature review alone cannot.

1. To investigate whether representing candidate vulnerabilities as a graph with explicit prerequisite relationships — rather than as an unordered list — can meaningfully improve how an autonomous agent chooses what to explore next. This includes formalizing such a structure (which we are currently calling a **Vulnerability Dependency Graph, or VDG**) to a level of precision sufficient for implementation and empirical testing.
2. To examine ways of separating what an agent has *confirmed* about a target from what it is merely *hypothesising*, so that discovery quality and planning quality can eventually be evaluated somewhat independently of each other.
3. To determine an appropriate multi-layer orchestration structure for the overall agent — drawing on the structural patterns that recur across the strongest-performing systems in the reviewed literature — and to develop this structure to the point where it is implementable.
4. To consider whether allowing an agent to retain and reuse strategies across multiple engagements offers a genuine advantage, or whether it primarily introduces risk (for example, a strategy that succeeded against one version of a technology being harmful against a different version).
5. To identify which existing, published benchmarks are appropriate for evaluating this class of system, and to begin drafting an evaluation plan around them rather than constructing new benchmarks independently.

## 1.4 Scope of the Research

Based on our reading of the literature so far, we intend to bound this project by one governing rule: we plan to work only with attack surfaces for which a dedicated, reusable, oracle-backed benchmark already exists in the reviewed literature, rather

than constructing new benchmarks ourselves. This decision keeps our evaluation grounded and reproducible. It is a scope we have provisionally settled on; it may narrow further once early implementation work gives us more information about what is achievable within the remaining timeline. Table 1.2 summarises the attack-surface families we are currently considering in scope and the benchmarks that would govern any claim made about them.

TABLE 1.2: Attack surfaces currently considered in scope, and their candidate governing benchmarks (preliminary).

| Attack surface                | Candidate benchmark(s)                                                                                                 | Representative vulnerability types                                                          |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Web application (HTTP/HTML)   | Fang et al. 15-CVE sandbox [5]; HPTSA 14-CVE zero-day suite [6]; CVE-Bench (40 CVEs) [1]; Pen-testEval (346 tasks) [2] | SQLi, XSS, CSRF, SSRF, SSTI, LFI, file-upload RCE, IDOR, authentication bypass, JWT forgery |
| GraphQL APIs                  | PrediQL 6-API suite [10]                                                                                               | Schema abuse, dependency-chain injection, batched-auth bypass, IDOR                         |
| Multi-host / Active Directory | Incalmo MHBench (40 environments) [11]                                                                                 | Lateral movement, credential reuse, privilege escalation                                    |

**What we currently intend to leave out.** General REST API exploitation is not something we plan to evaluate directly: no standardised, reusable REST API benchmark equivalent to CVE-Bench or PrediQL was found in the reviewed literature. Some underlying techniques from the REST API testing literature (dependency inference between API calls, response-feedback pruning) may still be useful internally, but we do not plan to report REST-API-specific results. Binary exploitation, physical- and network-layer attacks, and social engineering are also outside what we currently intend to attempt, for the same reason — the absence of a suitable existing benchmark within the reviewed literature.

## 1.5 Research Direction

We want to be careful not to overstate what this thesis has established at this point. None of the following has been implemented or evaluated. What follows is the direction we currently believe is worth investigating, framed as working hypotheses rather than results already in hand. We expect this framing to be revised as implementation and early experiments give us evidence the literature review alone cannot provide.

- **Dependency-aware exploration.** Our primary hypothesis is that combining exploration-style candidate selection with an explicit, dynamically-grown model of prerequisite relationships between vulnerabilities — rather than treating either concern in isolation — can improve how thoroughly and how sensibly an autonomous agent searches a target. We intend to test this against CVE-Bench and PentestEval once an implementation exists, but we are not yet in a position to state expected performance numbers with confidence.
- **Reuse of prior experience across missions.** We would like to examine whether allowing an agent to retain and selectively reuse strategies from earlier engagements produces a measurable benefit on targets that share underlying technology, without introducing the negative-transfer risk described in Section 1.6. Whether this ends up as a core research question or supporting infrastructure will depend on what early design work shows.
- **An evaluation grounded in existing benchmarks.** Independently of whether the above hypotheses hold, we intend for our evaluation to be run consistently across more than one existing, independently-published benchmark, with standardised oracles and clearly separated per-surface reporting — since we did not find this done consistently across the papers we reviewed.

Several further mechanisms that recur across the surveyed literature — structured communication between agent components, retry-and-diagnose validation loops,

and detailed logging — currently appear to be necessary engineering infrastructure for any implementation of the above rather than independent research questions in themselves. We plan to revisit this framing as implementation progresses.

## 1.6 Anticipated Challenges

Several challenges surfaced repeatedly while we were reading the literature, and we expect them to shape our work over the next six months regardless of which specific design choices we ultimately make. We are listing them here, at an early stage, rather than waiting to encounter them during implementation:

1. **Inferring dependencies without ground truth.** Any prerequisite structure our agent builds on its own will be inferred by an LLM rather than annotated by an expert, and we expect this to be the single riskiest part of whatever we build. Before relying on such a structure for any evaluation, we plan to first run a small pilot study against PentestEval’s expert-annotated dependencies to get a sense of how accurate our inference actually is.
2. **Sandbox results may not transfer to the real world.** Fang et al. [4] found only 1 exploitable XSS in 50 real-world candidate sites (2%), against 73.3% in a matched sandbox. We intend to keep this gap in mind throughout and to avoid presenting sandbox numbers as if they were real-world numbers.
3. **Some benchmarks are small.** PrediQL’s GraphQL suite, for instance, covers only 6 APIs — real and standardised, but far smaller than benchmarks like CVE-Bench. Any conclusion drawn from it will need to be stated as exploratory rather than definitive.
4. **Cost is a moving target.** Whatever cost-per-exploit figures we eventually report will be tied to a specific model and pricing at a specific point in time, and we plan to state that explicitly rather than presenting cost as a fixed property of the approach.



5. **Reused strategies could backfire.** A strategy that worked against one version of a piece of software could plausibly be actively harmful against a different version. We have not found this risk addressed in the papers we reviewed, which is one reason we consider it worth attention from the start.
6. **Sensitivity to configuration choices.** If our approach depends on several tunable parameters, we would want to check how sensitive results are to those choices rather than reporting numbers from a single, possibly fortunate, configuration.
7. **Being honest about what generalises.** If we do end up testing across more than one attack surface, we want to be careful about the difference between “a shared design” and “one literally unmodified system”, since we expect surface-specific components to be necessary either way.

## 1.7 Organisation of This Report

This report reflects the first stage of work on this thesis, and its organisation is intended as a plan for the full document we expect to submit in approximately six months, not a description of chapters that already exist in complete form. Chapter 2 presents a review of a focused selection of papers on autonomous VAPT, organised by architectural approach, and identifies the gaps in prior work that motivated the direction described in Sections 1.2 and 1.3; this chapter is the most developed at present. Chapter 3 will present our proposed system design as it develops — currently sketched at a high level and expected to be substantially revised as implementation proceeds. Chapter 4 will lay out our work breakdown, timeline, and milestones for the remaining months, and will be kept current as work progresses. Chapter 5 will specify and, eventually, report on our evaluation plan once an implementation exists to evaluate. Chapter 6 will summarise the completed work, and is not expected to be written in any substantive form until much later in the thesis timeline.

# Chapter 2

## Literature Review and Related Work

### 2.1 Domain Overview

This chapter reviews the recent literature on LLM-based autonomous penetration testing and identifies, at a preliminary level, the patterns and gaps that motivate the research direction described in Chapter 1. The review covers a focused selection of eleven papers, chosen as the most directly relevant to our research question from a broader set of papers read during the preparation of this report. Table 2.1 provides an overview of these eleven papers; the sections that follow discuss them in more depth.

Autonomous penetration testing — the use of LLM agents to independently discover and exploit weaknesses in a target system without step-by-step human direction — has emerged as an active research area only in the last two to three years. The domain sits at the intersection of two older fields: Vulnerability Assessment and Penetration Testing (VAPT), which traditionally relies on human operators following structured methodologies (reconnaissance, weakness identification, exploitation, and reporting), and LLM agent research, which studies how language models can plan, use tools, and act over multiple steps toward a goal. Our work, RedGrid, is

situated in this intersection: it is concerned with how an LLM-driven multi-agent system can carry out the VAPT workflow autonomously against realistic, benchmarked targets.

A useful way to frame the domain is around two capabilities that an autonomous VAPT agent must exhibit simultaneously. The first is *exploration*: the agent must search a large space of candidate weaknesses across an unfamiliar attack surface without being told in advance which one is exploitable. The second is *prerequisite reasoning*: many real exploits are not single actions but chains, where one weakness (for example, an information disclosure or a misconfigured endpoint) must be established before a second, more consequential weakness becomes reachable. As the literature reviewed in this chapter shows, most existing systems have been developed to strengthen one of these capabilities at the expense of the other — a pattern that motivates the closer reading in the sections that follow.

The sections that follow first survey the systems and benchmarks from this selection (Section 2.2), then draw out the recurring architectural patterns across them (Section 2.3), and finally identify, at a preliminary level, where the reviewed literature appears to leave a gap (Section 2.4).

## 2.2 Review of Existing Work

This section discusses the eleven papers selected in Table 2.1, grouped by the type of contribution they represent.

### 2.2.1 Early Single-Agent Approaches

The earliest work in this space used a single LLM agent operating in a Reason + Act (ReAct) loop against a target. Fang et al. [4] showed that a single GPT-4 agent with browser access could autonomously exploit simplified web vulnerabilities, establishing that the basic capability exists. A subsequent study by the same group [5] collected a benchmark of 15 real-world one-day vulnerabilities and found that

TABLE 2.1: The eleven papers selected for review, with their main focus and relevance to this research.

| No. | Paper                                                                                | Main Focus                                | Why Relevant                                                                                        |
|-----|--------------------------------------------------------------------------------------|-------------------------------------------|-----------------------------------------------------------------------------------------------------|
| 1   | Fang et al. (2024) — LLM Agents can Autonomously Hack Websites [4]                   | Single-agent web exploitation             | Foundational: first demonstration of autonomous web attacks; establishes sandbox vs. real-world gap |
| 2   | Fang et al. (2024) — LLM Agents can Autonomously Exploit One-day Vulnerabilities [5] | Single-agent CVE exploitation             | Shows GPT-4 can exploit real CVEs; key result: exploiting $\neq$ discovering                        |
| 3   | Zhu et al. (2024) — Teams of LLM Agents can Exploit Zero-Day Vulnerabilities [6]     | Hierarchical multi-agent VAPT             | First zero-day autonomous exploitation; introduces hierarchical planner + specialist design         |
| 4   | Deng et al. (2024) — Pen-testGPT [7]                                                 | Single-agent with reasoning/parsing split | Identifies context-loss failure mode in long sessions; motivates structured memory and handoff      |
| 5   | Kong et al. (2025) — VulnBot [8]                                                     | Multi-agent, specialised                  | Representative of team-dispatch architecture; shows flat dispatch without dependency modeling       |
| 6   | Wang et al. (2025) — CHECKMATE [9]                                                   | LLM agents + classical planning           | Explicit prerequisite modeling via PDDL; evaluated on curated scenarios only                        |
| 7   | Singer et al. (2025) — Incalmo [11]                                                  | Multi-host LLM red teaming                | Extends autonomous VAPT to multi-host / Active Directory environments; declarative task API         |
| 8   | Liu et al. (2025) — GraphQL API testing PrediQL [10]                                 | GraphQL API testing with LLMs             | Shows schema-based dependency reasoning for a non-web attack surface                                |
| 9   | Zhu et al. (2025) — CVE-Bench [1]                                                    | Benchmark for web CVE exploitation        | Provides empirical evidence that insufficient exploration is the dominant failure mode              |
| 10  | Yang et al. (2025) — Pen-testEval [2]                                                | Stage-level benchmark                     | Shows attack decision-making (prerequisite reasoning) is the weakest stage in current systems       |
| 11  | Wang et al. (2025) — Survey on LLM-based Autonomous Agents [3]                       | General LLM agent survey                  | Provides the conceptual and architectural vocabulary underlying all domain-specific systems         |

GPT-4, when given the CVE description, exploited 87% of them — while every other tested model and every open-source scanner achieved 0%. Crucially, without the CVE description, GPT-4’s success dropped to 7%, establishing an important early distinction: exploiting a vulnerability whose location is known is a very different capability from discovering an unknown vulnerability independently. This distinction motivates much of the subsequent work, and much of our own.

Deng et al. [7] extended single-agent design with an explicit reasoning/generation/parsing split aimed at reducing context loss over long testing sessions — a failure mode they document in detail, and that also appears as a concern across several subsequent systems. PentestGPT was evaluated on the HackTheBox and VulnHub platforms, which provide machine-level penetration testing challenges. Their work identifies a fundamental tension: agents need a long conversational history to maintain awareness of what they have already tried, but long histories inflate the context window and degrade reasoning quality. This tension between memory breadth and reasoning focus is one of the recurring architectural challenges in the literature.

### 2.2.2 Multi-Agent and Team-Based Orchestration

A second line of work replaces the single agent with a team of cooperating agents, motivated by the observation that a single flat loop struggles to scale to wide or unfamiliar attack surfaces. Zhu et al. [6] introduced a hierarchical planner (HPTSA) that dispatches task-specific sub-agents and was the first system reported to exploit real zero-day vulnerabilities autonomously — vulnerabilities for which no CVE description is provided. This result is significant: it shows that, with the right architecture, LLM agents can discover vulnerabilities rather than merely exploit known ones.

Kong et al. [8] propose VulnBot, a comparable multi-agent framework with explicit role specialisation — separating reconnaissance, planning, and exploitation into distinct agent roles. Like HPTSA, VulnBot dispatches tasks from a central planner to specialist agents, but does not explicitly model prerequisite relationships between

candidate vulnerabilities. The tasks are dispatched as a flat list, and agents work through them in sequence without a formal model of what depends on what.

A general point that emerges from these multi-agent systems — and that is supported by the broader survey of the field [3] — is that architecture, not model scale, appears to be the dominant variable in autonomous agent performance. A well-structured pipeline tends to outperform an unstructured loop even when the latter uses a more capable base model. This architectural primacy motivates our attention to the structural design of RedGrid.

### 2.2.3 Planning-Augmented Approaches

A smaller set of papers pairs LLM agents with a more classical planning component, aiming to give the agent an explicit model of prerequisites between actions rather than relying purely on the LLM’s implicit reasoning. Wang et al. [9] combine LLM agents with classical planning techniques (PDDL-style operators) for penetration testing, and report stronger performance on the specific scenarios where this planning is applicable. These systems typically require the relevant weaknesses to be enumerated in advance, however, which limits their applicability to open-ended discovery on a previously unseen attack surface — a limitation discussed further in Section 2.4.

### 2.2.4 Non-Web Attack Surfaces and Benchmarks

The final group of papers covers two non-web attack surfaces and the two benchmark studies that provide the quantitative evidence for the failure modes described in Chapter 1.

Singer et al. [11] extend autonomous penetration testing to multi-host and Active Directory-style network environments, where the agent must perform lateral movement, credential reuse, and privilege escalation across multiple networked hosts. This is a qualitatively different challenge from single-target web testing: the agent

must maintain awareness of its progress across an evolving network of compromised and uncompromised hosts. Incalmo addresses this partly through a structured, declarative task vocabulary that keeps the agent’s planning language manageable. Liu et al. [10] take a different direction, targeting GraphQL APIs specifically. Their work is notable for introducing schema-based dependency reasoning within the API testing context: a GraphQL schema encodes relationships between types and fields that the agent can exploit to guide its testing strategy, drawing on the same underlying intuition about prerequisite relationships that motivates our own work.

On the evaluation side, Zhu et al. [1] introduce CVE-Bench, a benchmark of 40 critical ( $CVSS \geq 9.0$ ) real-world web-application CVEs with oracle-backed pass/fail signals. Its failure-mode analysis provides the clearest available quantitative evidence that insufficient exploration — not reasoning quality — is the primary bottleneck in current agents. Yang et al. [2] introduce PentestEval, which decomposes the penetration testing pipeline into discrete stages and measures agent performance at each stage. Its finding that attack decision-making — reasoning about which weakness to pursue next, given what has been discovered so far — is the weakest stage provides complementary evidence to CVE-Bench: where CVE-Bench identifies exploration breadth as the failure, PentestEval identifies prerequisite reasoning as the failure, and together they suggest that the two capabilities need to be addressed jointly.

## 2.3 Comparative Analysis

Table 2.2 summarises the eleven reviewed systems and benchmarks along four dimensions that recurred throughout the literature review: whether the system uses a single agent or a coordinated team, whether it models dependencies between candidate weaknesses explicitly, whether it retains any form of memory across sessions, and which benchmark(s) it was evaluated against.

A few patterns are visible from this comparison. First, systems that scale to a wide or unfamiliar attack surface (HPTSA, VulnBot) dispatch tasks flatly, without an explicit model of which weaknesses depend on which others. Second, the one system

TABLE 2.2: Comparison of systems and benchmarks reviewed, across four recurring architectural dimensions.

| System Work     | / | Architecture                          | Dependency Modeling              | Cross-Session Memory | Benchmark(s)            |
|-----------------|---|---------------------------------------|----------------------------------|----------------------|-------------------------|
| Fang et al. [4] |   | Single agent (ReAct)                  | None                             | None                 | Simplified websites     |
| Fang et al. [5] |   | Single agent (ReAct)                  | None (given CVE desc.)           | None                 | 15 one-day CVEs         |
| HPTSA [6]       |   | Hierarchical planner + task agents    | None (flat dispatch)             | None                 | 15 zero-day CVEs        |
| PentestGPT [7]  |   | Single agent, split reasoning/parsing | Implicit (LLM only)              | None                 | HTB/VulnHub machines    |
| VulnBot [8]     |   | Multi-agent, role-specialised         | None (flat dispatch)             | None                 | HTB-style scenarios     |
| CHECKMATE [9]   |   | Agent + classical planner             | Explicit, pre-enumerated         | None                 | Curated scenarios       |
| Incalmo [11]    |   | Multi-host orchestration              | Partial (host/-credential graph) | None                 | MHBench (40 envs.)      |
| PrediQL [10]    |   | LLM-guided fuzzer                     | Schema-derived                   | None                 | 6 GraphQL APIs          |
| CVE-Bench [1]   |   | Benchmark (no agent)                  | N/A                              | N/A                  | 40 critical web CVEs    |
| PentestEval [2] |   | Benchmark (modular pipeline)          | Explicit (ground-truth injected) | None                 | 12 real-world scenarios |
| Wang et al. [3] |   | Survey                                | N/A                              | N/A                  | General (survey)        |

that does model dependencies explicitly (CHECKMATE) is evaluated on curated scenarios where relevant weaknesses are pre-enumerated, and it is not clear from the reviewed paper how well this approach would extend to open-ended discovery. Third, cross-session memory — the ability for an agent to retain and reuse what it learned in a previous engagement against a similar target — is absent from every system in the table. Finally, the two benchmark papers (CVE-Bench and PentestEval) each provide stage-level failure analysis that points in the same direction: existing systems are weakest precisely at the combination of broad exploration and structured prerequisite reasoning that would be needed to handle a wide, previously unseen attack surface.



These observations are preliminary, and the following section discusses their implications for the research direction this thesis is pursuing.

## 2.4 Research Gap

The literature reviewed in this chapter suggests, tentatively, that autonomous VAPT systems have so far been developed to address one of two capabilities at a time, rather than both simultaneously. Table 2.3 summarises this observation.

TABLE 2.3: Observed split in the reviewed literature between exploration-oriented and dependency-reasoning-oriented systems (preliminary reading).

| Area                                | What the reviewed literature shows                                                                                                                                 | Observed limitation / open area                                                                                         |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Exploration breadth                 | HPTSA and VulnBot scale to wide, unfamiliar attack surfaces using hierarchical team dispatch                                                                       | Neither models prerequisite relationships between discovered weaknesses; tasks are dispatched as flat lists             |
| Prerequisite / dependency reasoning | CHECKMATE introduces classical planning with explicit prerequisites; PentestEval shows that ground-truth ADM produces the largest single-stage improvement (+0.14) | Both require weaknesses to be pre-enumerated; applicability to open-ended discovery is unclear from the reviewed papers |
| Cross-session memory                | No system in the reviewed set retains and reuses strategies across multiple missions                                                                               | An open area that may warrant investigation, though its value relative to the above two concerns is currently unclear   |
| Cross-surface evaluation            | Every reviewed system is evaluated against a single attack-surface family                                                                                          | No system evaluates the same architecture across web, GraphQL, and multi-host surfaces simultaneously                   |

On one side, the reviewed literature shows that systems built for broad exploration of an unfamiliar attack surface — HPTSA, VulnBot, and similar team-dispatch architectures — do not appear to model prerequisite relationships between candidate weaknesses explicitly. They search widely, but without a formal notion of what must be established before what. On the other side, systems that do reason explicitly about such dependencies, including CHECKMATE’s classical-planning layer and PentestEval’s ground-truth attack decision-making configuration, are evaluated on

scenarios where the relevant weaknesses have already been enumerated in advance. It is not yet apparent from the reviewed papers how well these approaches would scale to open-ended discovery on a wide, previously unseen surface.

This split is echoed in how the two most detailed benchmark studies in this selection report their results. CVE-Bench’s own failure analysis attributes the majority of unsuccessful attempts on its hardest real-world CVEs to insufficient exploration rather than to reasoning quality. PentestEval’s stage-level breakdown finds that the attack-decision-making stage — reasoning about which weakness to pursue next given what has already been discovered — is consistently the weakest stage across the systems it evaluates. Taken together, these two findings point, at a preliminary stage of the review, at two sides of what may be the same underlying gap: the reviewed literature does not yet appear to contain a system that combines broad, open-ended exploration with an explicit, dynamically-constructed model of dependencies between discovered weaknesses.

It should be stressed that this is an early-stage reading of the literature. The gap described above is offered as a working hypothesis to guide the remainder of the thesis, rather than as a finalized problem statement. The architecture and methodology of RedGrid will be developed and justified more carefully as the work progresses.

## 2.5 Summary

This chapter has reviewed eleven papers directly relevant to autonomous, LLM-driven penetration testing, covering early single-agent systems, multi-agent orchestration frameworks, planning-augmented approaches, non-web attack surfaces, and the benchmark studies that provide the quantitative evidence for the field’s current limitations. A comparative reading of this literature points, tentatively, to a recurring split between systems optimised for broad exploration and systems optimised

for dependency-aware reasoning, with the reviewed literature not appearing to contain a system that addresses both simultaneously, and with few systems evaluated across more than one attack-surface family.

As this report reflects an early stage of the thesis, the survey above should be read as a snapshot of work completed so far rather than a closed literature review. Over the coming months, the review will be deepened — in particular around the planning-augmented systems and the question of cross-session memory — and the observations in Section 2.4 will be used to motivate a more precisely scoped research question and methodology as the thesis develops.

# Chapter 3

## Proposed Methodologies

### 3.1 System Overview

### 3.2 Requirement Analysis

### 3.3 Proposed Architecture

### 3.4 System Workflow

### 3.5 Tools and Technologies

# Chapter 4

## Execution Plan

### 4.1 Work Breakdown Structure

### 4.2 Project Timeline

### 4.3 Milestones

### 4.4 Resource Requirements

### 4.5 Risk Assessment

# Chapter 5

## Experimental Results

### 5.1 Evaluation Plan

# Chapter 6

## Conclusions

### 6.1 Summary

### 6.2 Expected Deliverables

### 6.3 Future Works

# Bibliography

- [1] Yuxuan Zhu, Antony Kellermann, Dylan Bowman, Philip Li, Akul Gupta, Adarsh Danda, Richard Fang, Conner Jensen, Eric Ihli, Jason Benn, Jet Geronimo, Avi Dhir, Sudhit Rao, Kaicheng Yu, Twm Stone, and Daniel Kang. CVE-Bench: A benchmark for AI agents’ ability to exploit real-world web application vulnerabilities. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *PMLR*, 2025.
- [2] Ruozhao Yang, Mingfei Cheng, Gelei Deng, Tianwei Zhang, Junjie Wang, and Xiaofei Xie. PentestEval: Benchmarking LLM-based penetration testing with modular and stage-level design. Preprint, 2025.
- [3] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhi-Yuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 2025. arXiv:2308.11432.
- [4] Richard Fang, Rohan Bindu, Akul Gupta, Qiusi Zhan, and Daniel Kang. LLM agents can autonomously hack websites. arXiv preprint arXiv:2402.06664, 2024.
- [5] Richard Fang, Rohan Bindu, Akul Gupta, and Daniel Kang. LLM agents can autonomously exploit one-day vulnerabilities. arXiv preprint arXiv:2404.08144, 2024. v2, cs.CR.
- [6] Yuxuan Zhu, Antony Kellermann, Akul Gupta, Philip Li, Richard Fang, Rohan Bindu, and Daniel Kang. Teams of LLM agents can exploit zero-day vulnerabilities. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2026. arXiv:2406.01637.



- [7] Gelei Deng, Yi Liu, Víctor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. PentestGPT: Evaluating and harnessing large language models for automated penetration testing. In *Proceedings of the 33rd USENIX Security Symposium (USENIX Security 24)*, pages 847–864, Philadelphia, PA, 2024. USENIX Association.
- [8] He Kong, Die Hu, Jingguo Ge, Liangxiong Li, Tong Li, and Bingzhen Wu. VulnBot: Autonomous penetration testing for a multi-agent collaborative framework. arXiv preprint arXiv:2501.13411, 2025.
- [9] Lingzhi Wang, Xinyi Shi, Ziyu Li, Yi Jiang, Shiyu Tan, Yuhao Jiang, Junjie Cheng, Wenyuan Chen, Xiangmin Shen, Zhenyuan Li, and Yan Chen. Automated penetration testing with LLM agents and classical planning. arXiv preprint arXiv:2512.11143, 2025.
- [10] Shaolun Liu, Sina Marefat, Omar Tsai, Yu Chen, Zecheng Deng, Jia Wang, and Mohammad A. Tayebi. PrediQL: Automated testing of GraphQL APIs with LLMs. arXiv preprint arXiv:2510.10407, 2025.
- [11] Brian Singer, Keane Lucas, Lakshmi Adiga, Meghna Jain, Lujo Bauer, and Vyas Sekar. Incalmo: An autonomous LLM-assisted system for red teaming multi-host networks. arXiv preprint arXiv:2501.16466, 2025.

# Appendix A: RedGrid Reference Material

This appendix provides full specifications for the REDGRID system that would exceed the length constraints of the main chapters, including the complete VDG node schema, benchmark suite details, UCB hyperparameter search configuration, and ablation condition pseudocode.

## .1 Full VDG Node Schema

Table 1 provides the complete field listing for a VDG node as defined in Chapter 3. Every field is populated by the Team Manager via `VDG_AddNode`; no Specialist writes directly to the VDG.

TABLE 1: Complete VDG Node Schema

| Field                      | Type              | Description                                                                                                                                                                                                                        |
|----------------------------|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>weakness_id</code>   | <code>str</code>  | Unique identifier (e.g. <code>sqli-001</code> )                                                                                                                                                                                    |
| <code>vuln_class</code>    | <code>enum</code> | <code>SQLi</code>   <code>XSS</code>   <code>CSRF</code>   <code>SSTI</code>   <code>LFI</code><br>  <code>AuthBypass</code>   <code>IDOR</code>   <code>RCE</code>  <br><code>GraphQL-Injection</code>   <code>LateralMove</code> |
| <code>attack_intent</code> | <code>str</code>  | Natural language description of the goal                                                                                                                                                                                           |

| Field                       | Type                   | Description                                                                                                                |
|-----------------------------|------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <code>prerequisites</code>  | <code>List[str]</code> | <code>weakness_ids</code> that must reach <code>EXPLOITED</code> before this node is eligible                              |
| <code>enables</code>        | <code>List[str]</code> | <code>weakness_ids</code> that become <code>ELIGIBLE</code> once this node is <code>EXPLOITED</code>                       |
| <code>status</code>         | <code>enum</code>      | <code>ELIGIBLE</code>   <code>IN_PROGRESS</code>   <code>EXPLOITED</code>   <code>INFEASIBLE</code>   <code>BLOCKED</code> |
| <code>w</code>              | <code>float</code>     | Cumulative UCB reward (sum of outcome scores)                                                                              |
| <code>n</code>              | <code>int</code>       | Times this node has been selected (initialised to 1 to avoid division by zero)                                             |
| <code>phi</code>            | <code>float</code>     | Promise score $\phi \in [0, 1]$ : LLM-assessed exploitability likelihood                                                   |
| <code>delta</code>          | <code>float</code>     | Task difficulty index $\delta \in [0, 1]$ ; 1 = maximally hard                                                             |
| <code>E_ord</code>          | <code>int</code>       | Ordinal evidence score $E_{\text{ord}} \in \{0, 1, 2, 3, 4, 5\}$ (see Table 2)                                             |
| <code>epss_prior</code>     | <code>float</code>     | EPSS score $\in [0, 1]$ from NVD/FIRST API; default 0.05 if no CVE match                                                   |
| <code>context_load</code>   | <code>float</code>     | Estimated Specialist context cost $C \in [0, 1]$                                                                           |
| <code>estimated_cost</code> | <code>float</code>     | Estimated token cost for this node's exploitation                                                                          |
| <code>retry_count</code>    | <code>int</code>       | Number of Specialist invocations on this node                                                                              |
| <code>max_retries</code>    | <code>int</code>       | Default 3; on exhaustion $\rightarrow$ <code>INFEASIBLE</code>                                                             |
| <code>path_score</code>     | <code>float</code>     | Score of best feasible path this node participates in                                                                      |

| Field                        | Type                   | Description                           |
|------------------------------|------------------------|---------------------------------------|
| <code>source_el_nodes</code> | <code>List[str]</code> | EL node IDs that seeded this VDG node |
| <code>last_updated</code>    | <code>timestamp</code> | Timestamp of last field mutation      |

## .2 Ordinal Evidence Score ( $E_{\text{ord}}$ ) Reference

Table 2 is the full rubric used by the Evaluation Agent to assign  $E_{\text{ord}}$  after every Specialist task. The value is parsed deterministically from constrained JSON output — not inferred from free-form text.

TABLE 2: Ordinal Evidence Score ( $E_{\text{ord}}$ ) Rubric

| $E_{\text{ord}}$ | Label               | Criteria                                                                                                            |
|------------------|---------------------|---------------------------------------------------------------------------------------------------------------------|
| 0                | Unseen              | Node not yet acted on                                                                                               |
| 1                | Tool ran / nothing  | Tool executed; target responded normally; no anomaly detected                                                       |
| 2                | Weak signal         | Anomalous response (different status code, timing difference) but ambiguous                                         |
| 3                | Clear indication    | Behavioural evidence consistent with vulnerability (e.g. SQL error message, reflected input) but not yet controlled |
| 4                | Confirmed behaviour | Controlled behaviour demonstrated (e.g. parameterised timing differential, payload executing in non-live context)   |
| 5                | Oracle-confirmed    | Per-surface oracle confirms successful exploitation                                                                 |

## .3 Benchmark Suite Summary

Table 3 summarises all eight benchmark tiers used in REDGRID’s evaluation. No benchmark was constructed for this work; every tier references a published, reusable target set.

TABLE 3: Full REDGRID Benchmark Suite

| Tier | Benchmark                   | Surface     | Size        | Role                                      |
|------|-----------------------------|-------------|-------------|-------------------------------------------|
| 0    | Fang et al. sandbox         | Web         | 15 CVEs     | CI regression flow                        |
| 0b   | HPTSA 14-CVE suite          | Web (0-day) | 14 CVEs     | Zero-day mode validation                  |
| 1    | PENTESTEVAL                 | Web         | 12 / 346    | Stage-level diagnosis<br>UCB tuning       |
| 2    | CVE-BENCH                   | Web         | 40 CVEs     | <b>Primary</b> metric<br>(pass@1, pass@5) |
| 2b   | MAPTA / HackWorld / Cybench | Web         | ~180        | Cross-benchmark generalisation            |
| 3    | PREDIQL                     | GraphQL     | 6 APIs      | Schema-coverage baselines                 |
| 4    | MHBENCH                     | Multi-host  | 40 envs     | Host-compromise credential-theft          |
| 5    | BOUNTYBENCH                 | Web (prod.) | 25 systems  | Dollar-value cost-per-exploit             |
| 6    | PentestGPT set + HTB S8     | Web         | 18 machines | Live-competition validation               |

## .4 UCB Hyperparameter Search Grid

The UCB formula (Chapter 3) has seven tunable parameters. Table 4 documents the search ranges and defaults. Grid search is performed on Tier 1 (PENTESTEVAL) only, holding out the CVE-BENCH split.

Results must be reported with sensitivity analysis:  $\pm 10\%$  perturbation from each optimal value while holding others fixed.

TABLE 4: UCB Hyperparameter Search Grid

| Parameter         | Meaning                              | Search Range               | Default |
|-------------------|--------------------------------------|----------------------------|---------|
| $C_{\text{expl}}$ | Exploration constant                 | $\{0.5, 1.0, 1.414, 2.0\}$ | 1.414   |
| $\alpha$          | Weight for promise $\phi$            | $[0.1, 0.5]$ step 0.1      | 0.3     |
| $\beta$           | Weight for difficulty $(1 - \delta)$ | $[0.1, 0.5]$ step 0.1      | 0.2     |
| $\gamma$          | Weight for $E_{\text{ord}}/5$        | $[0.1, 0.5]$ step 0.1      | 0.4     |
| $\kappa$          | Context-load penalty                 | $[0.05, 0.2]$ step 0.05    | 0.1     |
| $\lambda$         | EPSS prior weight                    | $[0.05, 0.3]$ step 0.05    | 0.15    |
| $\mu$             | Cost-awareness penalty               | $[0.05, 0.2]$ step 0.05    | 0.1     |

## .5 Ablation Condition A1 — Precise Pseudocode

This section provides the exact algorithmic distinction between Ablation A1 conditions (b) and (c), which is critical for interpreting whether the unified VDG adds value over a stacked approach.

**Condition (b) — UCB with dependency-constrained eligible set (pre-filter):**

```

eligible = [v for v in VDG.nodes
            if v.status == ELIGIBLE
            and all_prerequisites_satisfied(v)]
scores   = {v: UCB_score(v, N=len(eligible))
            for v in eligible}
selected = max(eligible, key=lambda v: scores[v])

```

**Condition (c) — Stacked: UCB over all nodes, post-filter by dependency (the key discriminant):**

```

all_nodes = VDG.all_unattempted_nodes()
scores    = {v: UCB_score(v, N=len(all_nodes))
            for v in all_nodes}
ranked    = sorted(all_nodes,
                    key=lambda v: scores[v],

```

```

        reverse=True)
filtered  = [v for v in ranked
            if all_prerequisites_satisfied(v)]
selected  = filtered[0]

```

If condition (d) (full VDG with path scoring) outperforms condition (c), the unified graph structure has value beyond dependency filtering alone. If (d)  $\approx$  (c), the contribution downgrades to dependency-aware UCB filtering — still a contribution, but a narrower claim. This gate must be reported honestly.

## .6 Prior Work Gap Summary

Table 5 consolidates the gap analysis from Chapter 2 into a compact reference table.

TABLE 5: Prior Work Gaps and REDGRID Fixes

| Gap                                             | Papers exhibiting          | RedGrid fix                                                 |
|-------------------------------------------------|----------------------------|-------------------------------------------------------------|
| Flat dispatch, no prerequisite modelling        | HPTSA, MAPTA, T-Agent      | VDG dependency-constrained frontier                         |
| Dependency planning on pre-curated sets only    | PENTESTEVAL SMP, CHECKMATE | Dynamic VDG_AddNode from Specialist discovery               |
| Node-level scoring misses multi-step chains     | EGATS                      | Path scoring: product of node scores $\times$ impact / cost |
| No failure propagation                          | All 29 surveyed papers     | BLOCKED propagation + frontier recomputation                |
| Exploration failure unaddressed architecturally | CVE-BENCH (diagnostic)     | Full-surface Recon + dependency-constrained frontier        |
| Session/multi-turn state loss                   | Fang et al., Pentest-GPT   | Auth/Session Specialist + Session Persistence Service       |

| Gap                                          | Papers exhibiting             | RedGrid fix                                                  |
|----------------------------------------------|-------------------------------|--------------------------------------------------------------|
| Context pollution from rolling history       | PentestGPT, D-CIPHER, VulnBot | Fresh context per Specialist invocation                      |
| Long-session context inflation               | PentestGPT Finding 4          | FullCompact: EL+AL-based reconstruction at 85%               |
| Raw LLM confidence overconfident in UCB      | EGATS                         | $E_{\text{ord}}$ ordinal scale (0–5) replaces raw confidence |
| Single-attempt validation: false positives   | All single-agent systems      | Diagnosis-Adapt-Cap loop (up to 3 retries)                   |
| Per-mission failure repetition               | All fresh-context designs     | Episodic Failure Memory (4th FAISS tier)                     |
| No cross-mission skill promotion             | AWE, PrediQL, VulnBot         | 3-tier memory + oracle-gated skill promotion                 |
| Single benchmark surface evaluated per paper | All 29 papers                 | Web + GraphQL + multi-host under one orchestrator            |